First in, first filled: an order book made of queues

Aarav Kyparlis (ak13728), Jayden Leung (jal9756)
CSCI-UA.0102-060, Preliminary Submission

Idea, and why it is effective (Quality)

We teach the queue through the place where it decides real money: a stock exchange's limit order book. Each of the 5 bid and 5 ask price levels is drawn as its own horizontal queue of order chips. Every chip shows its order id, size and arrival time, and each queue labels its **head** and **tail**. Adding a limit order animates `enqueue()`: the chip slides in at the tail. A market order "walks" to the best opposite price and trades with the head, one `dequeue()` at a time. Partial fills visibly shrink the head chip, and filled chips leave with a flash. A live Operations panel names each step (`asks[101].enqueue(#14) O(1)`, `level.dequeue() → #3 O(1)`) and highlights the matching pseudocode line. A trade tape and a depth chart record the result. Step, Back, Play, Speed and Reset let a student slow any order down to one operation per click.

Four features target understanding, not just display. Our goal: a newcomer sees within a minute that each price is a queue and matching is FIFO.

- **FIFO vs LIFO toggle.** A card replays the same market order on a copy of the book under both rules. Students see that LIFO (a stack) lets the newest order cut ahead of one that waited longer, which is why exchanges use FIFO.
- **Predict the fill.** Each market order pauses before its first trade, the student clicks the chip they think fills first, and the answer comes back with the reason (best price, then earliest arrival).
- **Checkout-line skin.** The same queues redrawn as supermarket lines, to anchor the idea for beginners.
- **Three guided scenarios:** same price, partial fill, sweeping several levels. Each step has a one-line caption.

Novelty

The closest illustrations teach the queue in isolation. VisuAlgo's Linked List / Stack / Queue / Deque module [1] and the USF Linked List Queue visualization [2] animate enqueue and dequeue on a single list of integers, with step and replay controls. OpenDSA's Linked Queues chapter [3] uses slideshows and click-through exercises on the same abstract queue. On the finance side, the interactive order-book article by nikhedonia [4] simulates prices but states that it does not track order arrival time, so it has no time priority. Our project sits between the two: **many queues embedded in a real system**, with **price-time priority** as the thing being taught. It also adds a direct **FIFO-vs-LIFO contrast** on identical inputs and a **predict-then-check** mode. We found none of these four in any of the tools above.

Current status

Built and working: the pure TypeScript engine and its 16 passing unit tests. The full UI: the ladder of queues with head/tail markers, tail-entry and head-exit animations, partial-fill shrink, market-order walk, Operations panel with highlighted pseudocode, trade tape, depth chart, FIFO/LIFO comparison card, checkout skin, Predict mode, three scenarios, and deterministic Step/Back/Play/Speed/Reset. Keyboard controls, visible focus and reduced-motion support are in place. We checked the layout in a browser at 1440, 1024 and 375 px wide, and we built and served the production build locally under a GitHub Pages sub-path.

To do before Dec 1: publish the repository and deploy to GitHub Pages (the workflow is written but has not run yet). Run a 60-second comprehension test with classmates who have never seen an order book, then tighten the captions. Test with a screen reader and in Firefox and Safari. Add an animated "replay this under LIFO" step and browser-level UI tests.

Tools and techniques (Technicality)

TypeScript, React 19, Vite, Tailwind CSS 4, Motion (layout and enter/exit animations), and Radix accessible primitives. **The engine is pure TypeScript and UI-independent.** Each price level is a doubly linked `OrderQueue` with O(1) enqueue/dequeue/peek. Every operation returns an **event log**, with a snapshot of the book after each event, and the UI only plays that log. This makes playback deterministic: Step, Back and Reset are cursor moves, and the same inputs always produce the same animation. **Vitest** covers FIFO order, partial fills, multi-level sweeps, price priority across levels, the LIFO contrast and replay determinism. **GitHub Actions** tests, builds with the repository's base path, and deploys to Pages. We learned to keep a data structure separate from its animation (event sourcing), how layout animation works, and accessibility basics: no colour-only cues, reduced motion, ARIA live regions.

Links

Live site: to be added. Repository: to be added. [1] visualgo.net/en/list [2] cs.usfca.edu/~galles/visualization/QueueLL.html [3] opendsa-server.cs.vt.edu/ODSA/Books/CS2/html/QueueLinked.html [4] nikhedonia.net/articles/order-book-model